

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Engineering and Technology

Devang Patel Institute of Advance Technology & Research

Department Of Computer Science & Engineering

A.Y. 2026-2027[Odd]

Lab Manual

CEUE203: Object Oriented Programming.

Name: Vaidehi Parmar

ID: 25DCS073

Practical-6:

PART A:

1. Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.

Solution:

```
package prac61;
interface Switchable {
    void on();
    void off();

    default void toggle() {
        on();
    }
}
class Fan implements Switchable {
    private boolean status = false;

    public void on() {
        status = true;
        System.out.println(x: "Fan is ON");
    }

    public void off() {
        status = false;
        System.out.println(x: "Fan is OFF");
    }
}
class Light implements Switchable {
    private boolean status = false;
    public void on() {
        status = true;
        System.out.println(x: "Light is ON");
    }
    public void off() {
        status = false;
        System.out.println(x: "Light is OFF");
    }
}
interface SwitchPolicy {
    boolean maySwitchOn(Switchable device, int hour);
}

public class main {
    Run | Debug
    public static void main(String[] args) {

        Switchable[] devices = {
            new Fan(),
            new Light()
        };

        for (Switchable device : devices) {
            device.toggle();
        }
        SwitchPolicy policy1 = new SwitchPolicy() {
            public boolean maySwitchOn(Switchable device, int hour) {
                return hour >= 6 && hour <= 22;
            }
        };
        SwitchPolicy policy2 = (device, hour) -> hour >= 8 && hour <= 20;
        System.out.println(policy1.maySwitchOn(devices[0], hour: 10));
        System.out.println(policy2.maySwitchOn(devices[1], hour: 23));
    }
}
```

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac prac61/main.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java prac61/main
Fan is ON
Light is ON
true
false
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```

Problem Statement:

2. Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.

Solution:

```java
package prac62;
interface Notifier {
    void send(String message);
}
interface Urgent {
}
class UrgentNotifier implements Notifier, Urgent {
    private final Notifier notifier;

    UrgentNotifier(Notifier notifier) {
        this.notifier = notifier;
    }
    public void send(String message) {
        notifier.send(message);
    }
}
public class main {
    Run | Debug
    public static void main(String[] args) {
        Notifier email = message ->
            System.out.println("Email: " + message);
        Notifier sms = message ->
            System.out.println("SMS: " + message);
        Notifier urgentEmail = new UrgentNotifier(email);
        Notifier[] senders = {
            urgentEmail,
            sms
        };
        String message = "Your assignment is due tomorrow.";
        for (Notifier sender : senders) {
            sender.send(message);
            if (sender instanceof Urgent) {
                sender.send(message);
            }
        }
    }
}
```

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac prac62/main.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java prac62/main
Email: Your assignment is due tomorrow.
Email: Your assignment is due tomorrow.
SMS: Your assignment is due tomorrow.
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```

Problem Statement:

Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR.

Solution:

DiscountRule.java

```java
package DiscountEngine.src.discount;

@FunctionalInterface
public interface DiscountRule {
    double apply(double price);
}
```

Main.java

```java
package DiscountEngine.src.app;
import DiscountEngine.src.discount.DiscountRule;
import java.util.*;
public class Main {
    Run | Debug
    public static void main(String[] args) {

        List<Double> prices = Arrays.asList(
            ...a: 1000.0,
            2500.0,
            5000.0,
            7500.0
        );
        Scanner sc = new Scanner(System.in);
        System.out.println(x: "Choose discount:");
        System.out.println(x: "1. 10% discount");
        System.out.println(x: "2. 20% discount");
        System.out.println(x: "3. Flat ₹500 discount");
        int choice = sc.nextInt();
        DiscountRule rule;
        switch (choice) {
            case 1:
                rule = price -> price * 0.90;
                break;

            case 2:
                rule = price -> price * 0.80;
                break;

            case 3:
                rule = price -> Math.max(a: 0, price - 500);
                break;

            default:
                rule = price -> price;
        }

        for (double price : prices) {
            System.out.println(
                "Original: " + price +
                " -> Final: " + rule.apply(price)
            );
        }

        sc.close();
    }
}
```

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac -d . DiscountEngine\src\discount\DiscountRule.java DiscountEngine\src\app\Main.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java DiscountEngine.src.app.Main
Choose discount:
1. 10% discount
2. 20% discount
3. Flat ?500 discount
2
Original: 1000.0 -> Final: 800.0
Original: 2500.0 -> Final: 2000.0
Original: 5000.0 -> Final: 4000.0
Original: 7500.0 -> Final: 6000.0
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```